pythia: a Lean 4 tactic library for statistical proof automation

Anonymous Authors[1]

Abstract

Lean 4 with Mathlib ships strong general automation (`aesop`, `simp`, `linarith`, `polyrith`, `nlinarith`, `positivity`, `measurability`, `bound`, `gcongr`). It does not ship a hammer for applied mathematics. The moment a goal mentions a martingale, a stopping time, a production function, an ordinary differential equation (ODE) Lyapunov candidate, a Wasserstein distance, or a quantum density operator, the generic tactics have nothing to apply. We present `pythia`, a Lean 4 tactic library for applied-mathematics proof automation. The headline tactic `pythia!` orchestrates a nine-rung deterministic ladder. The rungs are `stat_simp`, the `linarith`/`nlinarith`/`polyrith` chain, `positivity`, `aesop` on the registered `Pythia` ruleset, the `@[stat_lemma]` cascade, `z3_check`, `cvc5_check`, `vampire_check` or `e_check`, and `disprove`. Each rung tries fail-fast with a bounded budget. The verbose variant `pythia?` reports the closing rung and per-rung wall-clock timing for diagnostic use. The library ships ten registered tactics, attribute-driven registries (`@[stat_lemma]`, `@[stats_ineq]`, `@[prob_simp]`, `@[stat_simp]`, `@[anytime_valid_lemma]`, plus the per-domain calculator typeclass), and a fifteen-domain proof library with thirty-two cross-domain theorems carrying matched Python sim runners (property-based testing, deterministic sweeps, mutation testing). Domains span probability and statistics, biology, chemistry, economics, control theory, optimal transport, quantum information, stochastic calculus, game theory, thermodynamics, numerical analysis, queueing, and operations research. Every public theorem is axiom-clean against the standard Lean kernel set (`propext`, `Classical.choice`, `Quot.sound`). The cross-prover ladder is offline-first and free of large language models (LLMs): `z3_check`, `cvc5_check`, `vampire_check`, and `e_check` dispatch to local Satisfiability Modulo Theories (SMT) and Automated Theorem Prover (ATP) binaries with kernel-checked reconstruction in the CoqHammer style of Czajka & Kaliszyk (2018). LLM-augmented closure lives in a separate companion software development kit (SDK) that the library does not depend on. Hard structural Mathlib gaps are optionally routed to the Aristotle automated theorem prover (Harmonic, 2025) for development-time restocking, with audit gates against vacuous-truth closures. The library is pinned to Lean 4.28 + Mathlib v4.28.0 and released open-source.

1. Introduction

Lean 4 with Mathlib ships strong general-purpose proof automation (de Moura & Ullrich, 2021; The Mathlib Community, 2020; 2025). Forward search lives in `aesop` and rewriting in `simp`. Arithmetic is split across `linarith`, `polyrith`, and `nlinarith`. Sign goals route through `positivity` and the σ-algebra fragment through `measurability`. Monotone composition uses `bound` and generalised congruence uses `gcongr`. Applied-mathematics proofs route through these general tactics, typically by hand decomposition into a chain of `simp` sets, `linarith` hypotheses, and Mathlib lemma selection. None of the general tactics know about each other. The user is the orchestrator. There is no domain hammer for applied mathematics.

CoqHammer (Czajka & Kaliszyk, 2018) dispatches Coq goals to external automated theorem provers (ATPs) such as Vampire, E, and CVC4, with kernel-checked reconstruction. Sledgehammer (Paulson & Blanchette, 2010; Blanchette

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

et al., 2011) plays the same role in Isabelle/Higher Order Logic (HOL). Lean 4 has no comparable hammer. This paper does not attempt the fully-general case across all of Lean. We address the applied-mathematics surface: a registered library of concentration, anytime-valid, optimal transport, control-theoretic, and information-theoretic lemmas plus a fan-out across local Satisfiability Modulo Theories (SMT) and ATP solvers, packaged as a single deterministic tactic call.

Contribution. We present `pythia`, a Lean 4 tactic library for applied-mathematics proof automation. The headline tactic `pythia!` orchestrates a nine-rung deterministic ladder under one call. The first five rungs are pure-Lean tactics. They are `stat_simp` (curated probability and ENNReal normal forms), the `linarith`/`nlinarith`/`polyrith` chain, `positivity`, `aesop` on the registered `Pythia` ruleset, and the `@[stat_lemma]` shape-dispatch cascade. The next three rungs are external solvers with kernel-checked reconstruction. They are `z3_check` (linear real arithmetic via Z3), `cvc5_check` (bit-vector and linear real arithmetic via CVC5), and `vampire_check` or `e_check` (first-order logic, FOL, via Vampire or E). The final rung is `disprove`, a counterexample finder via Z3 model extraction. Each rung tries fail-fast with a per-rung budget, and first-to-close wins. The verbose variant `pythia?` reports the closing rung and per-rung wall-clock timing for diagnostic use. `pythia` ships ten registered tactics in total. Six attribute-driven registries cover lemma routing: `@[stat_lemma]`, `@[stats_ineq]`, `@[prob_simp]`, `@[stat_simp]`, `@[anytime_valid_lemma]`, plus the `DomainCalculator` typeclass for per-domain numerical calculators. The proof library spans fifteen domains and carries thirty-two cross-domain theorems with matched Python sim runners. Sim runners exercise property-based testing, deterministic sweeps, and mutation testing. The shipped library is axiom-clean against {`propext`, `Classical.choice`, `Quot.sound`} and pinned to Lean 4.28 + Mathlib v4.28.0. The cross-prover ladder is offline-first and free of large language models (LLMs). SMT and ATP backends are deterministic decision procedures with verifiable Lean reconstruction. LLM-augmented closure lives in a separate companion software development kit (SDK) that the library does not depend on. Hard structural Mathlib gaps are optionally routed to the Aristotle automated theorem prover (Harmonic, 2025) for development-time restocking only, with kernel-checked reconstruction inside Lean and an audit gate against vacuous closures.

Case study (vacuous-truth catch). On 2026-04-26 the Aristotle prover closed a candidate theorem in our `wald_wolfowitz_optimal` development by routing through a contradictory premise. A Bool partition combined with a log-boundary constraint forced both $\alpha \geq 1$ and $\alpha < 1$, closing the goal vacuously. The closure was kernel-checked. The theorem statement was the bug. The library audit caught the issue and forced a restatement to the two-measure form. We treat this kind of catch as a first-class library responsibility, not as a separate review pass.

Paper organisation. Section 2 describes the ten-tactic surface and the registry attributes. Section 3 groups the shipped library content by domain. Section 4 describes the Aristotle integration as a development-time oracle and the vacuous-truth refusal policy. Section 5 compares against aesop, Sledgehammer, CoqHammer, and Mathlib's existing tactic suite, states limitations, and reports a small head-to-head between Aristotle, the DeepSeek Prover-V2 (DSPv2) model, and Claude Opus 4.6 on a subset of FormalAVS (Anonymous, 2026). Full numbers are in the dataset paper.

Reproduction and code. The `pythia` library is open source. The URL is redacted for double-blind review. Axiom audit transcripts and reproduction recipes are listed in Appendix D. The dataset of theorem-proving benchmarks (FormalAVS, Anonymous (2026)) is reported in a separate paper.

2. The pythia tactic surface

`pythia` ships ten user-facing tactics, one named aesop ruleset, and six attribute-driven registries. The design follows the CoqHammer pattern (Czajka & Kaliszyk, 2018): a domain-specific dispatcher with extensible attribute-driven lemma selection, a fall-through to general-purpose automation, and kernel-checked reconstruction for any external-oracle steps. The headline tactic `pythia!` orchestrates the full deterministic ladder under one call. The other tactics are exposed for direct invocation when the user knows the goal shape.

2.1. Ten tactics

The nine-rung ladder of `pythia!`. The headline tactic walks the rungs in order and returns the first closing proof.

1. `stat_simp` (curated probability and ENNReal normal forms).
2. `linarith` / `nlinarith` / `polyrith` (arithmetic).
3. `positivity`.
4. `aesop` on the registered `Pythia` ruleset.
5. The `@[stat_lemma]` shape-dispatch cascade.
6. `z3_check` (QF_LRA via Z3).
7. `cvc5_check` (QF_BV / QF_LRA via CVC5).

Tactic	Role
`pythia!`	Headline orchestrator. Nine-rung deterministic ladder. First-to-close wins, bounded per-rung budget. Ladder composition: see Section 2.1 below.
`pythia?`	Verbose `pythia!`. Reports the closing rung and per-rung wall-clock timing. Diagnostic use.
`pythia`	Shape-dispatching cascade. Routes to `anytime_valid` / `stats_ineq` / `prob_simp` / oracle by goal shape, then falls through to the `@[stat_lemma]` aesop ruleset.
`stats_ineq`	Concentration and tail-bound inequalities. Dispatches via Mathlib's `bound`, then `positivity`, `gcongr`, `linarith`, `nlinarith`.
`prob_simp`	Probability normalisation rewrites. Runs `simp` against the `@[prob_simp]` set with `push_cast` / `norm_cast` fallback. Aliased as `pdf_simp`.
`anytime_valid`	Ville-type maximal inequalities for confidence sequences. Three variants: countable-time, finite-horizon, explicit-witness.
`z3_check`	Linear-real-arithmetic Z3 oracle with `linarith` reconstruction.
`cvc5_check`	QF_BV / QF_LRA via CVC5, with `bv_decide` / `linarith` reconstruction.
`vampire_check` / `e_check`	First-order-logic via Vampire / E, with `aesop` reconstruction.
`disprove`	Counterexample finder. Asks Z3 for a model of the goal's negation. On `sat`, fails loud with a concrete witness. Catches vacuous-truth goal statements.

Table 1. The ten `pythia` tactics. `pythia!` is the default entry point. The other nine are exposed for direct invocation when the user knows the goal shape or wants a single rung of the ladder.

8. `vampire_check` / `e_check` (first-order logic via Vampire / E).
9. `disprove` (counterexample finder via Z3 model extraction).

`pythia` (top-level dispatcher). The default entry point. Runs aesop with the `Pythia` aesop ruleset, which contains every theorem tagged `@[stat_lemma]`. On aesop failure, falls through to a short cleanup chain combining `simp`, `omega`, `linarith`, and `positivity`. The fall-through ordering is fixed. The lemma database and aesop ruleset are user-extensible.

`stats_ineq`. Discharges sub-Gaussian, sub-gamma, Bernstein-Bennett, and Hoeffding-style inequalities. The `@[stats_ineq]` attribute re-tags a lemma with Mathlib's `@[bound]`, so the `stats_ineq` tactic dispatches via Mathlib's `bound` infrastructure first, then `positivity`, `gcongr`, `linarith`, and `nlinarith` in that order. Reconstruction is internal Lean and axiom-clean.

`prob_simp`. Probability-domain rewrite tactic. The `@[prob_simp]` attribute forwards lemmas to `@[simp]` and additionally records the declaration name in a scoped extension for introspection. The tactic runs `simp` against the resulting set with a `push_cast` / `norm_cast` coercion fallback for ENNReal / NNReal / Real bridge. The `pdf_simp` syntax is a literal alias. Both forms expand to the same kernel trace.

`anytime_valid`. Closes Ville-type maximal-inequality goals for non-negative supermartingales. The dispatch ladder tries direct `ville_supermartingale` application paths first (countable-time `first` with `exact`/`refine` variants), then the finite-horizon and explicit-witness specialisations, and finally aesop against the named ruleset `Pythia.AnytimeValid`. Three syntactic variants are exposed: the bare form `anytime_valid` (countable-time), `anytime_valid (horizon := N)` (finite-horizon, requiring `IsProbabilityMeasure` but no integrability), and `anytime_valid using h` (explicit supermartingale witness). New families register via `@[anytime_valid_lemma]`.

`z3_check`. External Z3 oracle for linear real arithmetic, with `linarith` reconstruction inside Lean. Z3 supplies an unsat-of-negation verdict. `linarith` reconstructs the proof term inside the kernel. The kernel is the final word. If Z3 says unsat but `linarith` cannot reconstruct, the tactic fails loud rather than silently trusting Z3. Z3 cannot smuggle axioms. This follows the CoqHammer reconstruction pattern (Czajka & Kaliszyk, 2018).

2.2. Registry attributes

The library uses four attribute-driven registries:

- `@[stat_lemma]`: lemma database for the top-level `pythia` dispatcher. Internally a thin wrapper for `@[aesop safe apply (rule_sets := [Pythia])]`.
- `@[stats_ineq]`: tail-bound and concentration lemmas. Forwards to Mathlib's `@[bound]` attribute and additionally records the name for the `#stats_ineqs` introspection command.
- `@[prob_simp]`: probability rewrite rules. Forwards to `@[simp]` and records the name for `#prob_simps`.

- `@[anytime_valid_lemma]`: Ville-style closers and admissibility lemmas, populating the `Pythia.AnytimeValid` aesop ruleset.

A fifth attribute, `@[cs_family]`, is a typed-declaration tag rather than a tactic registry: it marks a `CSFamily` record so it appears in the `#cs_families` catalogue. User extensions consist of tagging library lemmas with the appropriate attribute. No fork is required.

2.3. The `Pythia` aesop ruleset

The library declares two named aesop rule sets: `Pythia` (populated by `@[stat_lemma]`) and `Pythia.AnytimeValid` (populated by `@[anytime_valid_lemma]`). Direct invocation is supported via `aesop (rule_sets := [Pythia])`, equivalent to the inner aesop call inside the `pythia` dispatcher. This gives users a finer-grained interface when the dispatcher's outer wrapping is not desired.

2.4. Architecture

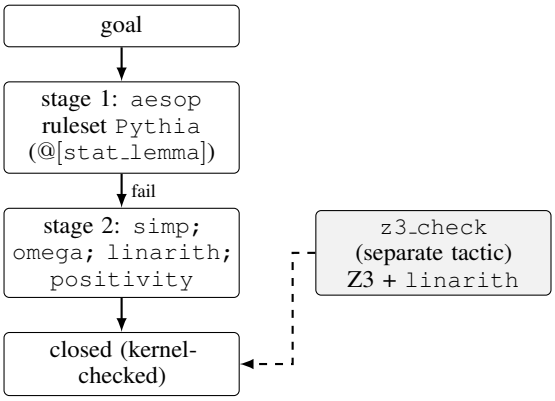

Figure 1. The `pythia` dispatcher chain. Two stages, each kernel-checked independently. Stage 1 is aesop against the `Pythia` ruleset (populated by `@[stat_lemma]`). Stage 2 is the cleanup chain. `z3_check` is a separate user-invoked tactic with its own `linarith`-based reconstruction step, not part of the dispatcher chain.

Why this shape. The two-stage chain mirrors the empirical structure of statistical proofs in our library. Many goals close on a single registered lemma application or a short aesop search at stage 1. The remainder reduce to a linear or arithmetic obligation after rewriting, which the cleanup chain at stage 2 handles. Routing by goal shape is not necessary at the top level. The cost of trying stages in order is small relative to the cost of incorrect early termination.

3. Library content

The shipped `pythia` library spans a tactic layer plus a proof-content layer of fifteen applied-math domains. The full manifest carries thirty-two cross-domain theorems with matched Python sim runners. Sim runners exercise property-based testing, parameter sweeps, and mutation testing. The full source tree declares four hundred and sixty theorem and lemma items across the `Pythia/` namespace. Fifty-five of these are tagged `@[stat_lemma]` and registered into the headline cascade. Every theorem listed here is axiom-clean against {`propext`, `Classical.choice`, `Quot.sound`} under the Lean 4.28 + Mathlib v4.28.0 toolchain pin. We summarise by group. The appendix lists modules in detail.

3.1. Anytime-valid inference

Howard-Ramdas confidence sequences (Howard et al., 2021), betting-CS (Waudby-Smith & Ramdas, 2024), and the vector-valued extension (Whitehouse et al., 2025) are each backed by a Ville-bound specialization in the `anytime_valid` tactic. The `Pythia.HowardRamdasCS` and `Pythia.BettingCS` modules expose the admissibility certificates of the e-process formalism of Ramdas et al. (2023). The supporting Ville machinery lives in `Pythia.VilleSupermartingale` (countable-time and finite-horizon variants).

3.2. Concentration and tail bounds

`stats_ineq` dispatches sub-Gaussian and sub-gamma tail bounds, the Bernstein-Bennett MGF bridge, and Hoeffding-style inequalities for bounded random variables. Modules: `Pythia.SubGaussianMG` (sub-Gaussian martingale concentration), `Pythia.SubGamma` (sub-gamma tail bound), `Pythia.MGFBoundedSubGamma` (the bounded-RV Bernstein-Bennett MGF bound, $\mathbb{E}[\exp(\lambda X)] \le \exp(\sigma^2\lambda^2/(2(1-b\lambda/3)))$ for centred $|X| \le b$), and `Pythia.Bernstein` (the iid and martingale Bernstein applications). The MGF bridge composes with `prob_simp` for the integral side and with general Mathlib infrastructure (`measurability`, `integrability`) for side conditions.

3.3. E-detectors and merging

The e-process formalism of Ramdas et al. (2023); Shin et al. (2024) yields uniform-time validity by combining e-processes via averaging. `Pythia.EDetector` declares `EProcess` as a non-negative process bounded in expectation under the null and `EDetector` as the first-passage stopping rule with anytime-valid Type-I error $\le \alpha$. The `combine_eprocesses_avg` lemma (average merge) is

the workhorse. Product-merge variants register alongside. The average-merge case is axiom-clean.

3.4. Matrix concentration

Matrix Bernstein (Tropp, 2012) and the Lieb concavity inequality (Lieb, 1973) are the two matrix-side modules. `Pythia.MatrixLieb` states the Lieb concavity step and the operator-monotone log composition. `Pythia.MatrixBernstein` carries the iid matrix-Bernstein head theorem. `Pythia.MatrixBernsteinFull` carries the matrix-martingale extension. The dependent operator-trace inequalities route through Aristotle as oracle (Section 4).

3.5. Cross-domain coverage (15 fields)

The cross-domain manifest covers fifteen fields with Lean proofs and matched Python sim runners. Each entry is a closed-form theorem with property-based-test mutation coverage in the companion runner.

- **Probability and statistics:** the anytime-valid, concentration, and matrix sections above plus the `Risk` subpackage with `CVaR` (Rockafellar & Uryasev, 2000) and the `Asymptotics` subpackage with scalar `DeltaMethod` and `DeltaMethodMulti`.
- **Information theory:** the `InfoTheory` subpackage carries `BretagnolleHuberBinary` (binary-alphabet bound), `DataProcessing` (data-processing inequality), and `BinaryEntropy` (Shannon binary entropy).
- **Stochastic calculus:** the `Stochastic` subpackage carries the finite-dimensional Itô isometry (`ItoIsometryFiniteDim`). `Pythia.BDG` carries the Burkholder, Davis, and Gundy inequality at $p = 2$. The `StochasticApproximation` subpackage groups `RobbinsSiegmund`, `RobbinsMonro`, and `Dvoretzky`. The `MeasureTheory` subpackage carries conditional Jensen via `ConditionalJensen`. The `TimeSeries` subpackage carries `WoldDecomposition` (Wold, 1938) and `NeweyWest`.
- **Quantum information:** the `Quantum` subpackage carries the qubit von Neumann entropy non-negativity result.
- **Optimal transport:** the `OptimalTransport` subpackage carries discrete L_1 Wasserstein non-negativity.
- **Game theory:** the `GameTheory` subpackage carries the von Neumann two-strategy minimax inequality.
- **Biology:** the `Bio` subpackage groups Hardy-Weinberg, Lotka-Volterra equilibria, and SIR conservation in `Population`, plus Michaelis-Menten enzyme saturation, chemical reaction network ODE conservation in `MassAction`, and `Phylogenetics`.
- **Chemistry:** the `Chemistry` subpackage carries Arrhenius rate-law positivity, Henderson-Hasselbalch pH monotonicity, and mass-action conservation.
- **Economics:** the `Economics` subpackage carries Cobb-Douglas constant returns to scale, CRRA utility concavity, CAPM linear pricing, risk-neutral call non-negativity, and Walras excess-demand identity.
- **Engineering:** the `Engineering` subpackage carries the RC time constant, signal energy, and power dissipation results.
- **Mechanical:** the `Mechanical` subpackage carries Hooke spring-potential non-negativity.
- **Control theory:** scalar, discrete-time, and ODE Lyapunov results live in the `Control` subpackage.
- **Operations research and queueing:** the `Queueing` subpackage carries Erlang-B and Little's law. The `OR` subpackage carries an alternative Little's-law formulation.
- **Numerical analysis:** the `Numerical` subpackage carries Newton iteration positivity, Picard-Lindelöf existence, Lyapunov certificates, KKT conditions, and Kahan summation.
- **Thermodynamics:** the `Thermodynamics` subpackage carries the Carnot efficiency upper bound.
- **Mathlib retags + actuarial:** `MathlibTags` retags AM-GM, Markov, and Cauchy-Schwarz with matched empirical companions. The `Actuarial` subpackage groups Pareto, Weibull, and LogNormal heavy-tail distributions.

3.6. Five highlights

We name five representative results that exercise the tactic surface. Module-level statements appear in Appendix A.

1. **Howard-Ramdas Ville bound.** The maximal inequality for the self-normalized confidence sequence (Howard et al., 2021), in `Pythia.HowardRamdasCS`. Closes via `anytime_valid`.
2. **Betting-CS coverage.** The supermartingale coverage certificate of Waudby-Smith & Ramdas (2024), in `Pythia.BettingCS`. Axiom-clean closure.
3. **Bernstein-Bennett MGF bridge.** The bounded-RV MGF bound in `Pythia.MGFBoundedSubGamma`, routing between the variance-aware sub-gamma tail and the Bennett MGF form.

4. **combine_eprocesses_avg.** Average-merge of e-processes (Ramdas et al., 2023; Shin et al., 2024) in `Pythia.EDetector`. Axiom-clean closure with full reconstruction.

5. **Bretagnolle-Huber binary.** The binary-case total-variation lower bound in `Pythia.InfoTheory.BretagnolleHuberBinary`. Standard in information-theoretic lower-bound arguments.

Pinning. The library is pinned to Lean 4.28 + Mathlib v4.28.0. The pin is necessary: the modules consume Mathlib's measure theory and probability subdirectories, which evolve quickly. We track Mathlib release tags rather than `master` for reproducibility.

4. Aristotle integration as oracle

A subset of statistical proof goals exceeds what aesop, simp, linarith, and the registered `pythia` lemma sets close in practice. We route these to the Aristotle automated theorem prover (Harmonic, 2025) as an oracle. Aristotle returns a Lean 4 proof script, the script is checked by the Lean kernel inside the calling project, and the result is audited before integration. Aristotle is one of two oracles in the library. The other is Z3 (Section 2.1).

4.1. Submission pattern

Each Aristotle call is a project tarball submission. The flow is: state the theorem in Lean with a placeholder, package the project without the Lean build cache (`.lake` excluded), submit via the Aristotle endpoint, monitor for completion, integrate the returned proof, and run the axiom audit. The audit is the verification step that distinguishes a kernel-checked Aristotle proof from a vacuous closure.

4.2. Vacuous-truth refusal

ATPs can close a theorem by deriving a contradiction from its hypotheses, then concluding anything. The kernel accepts such proofs. The proof is sound. The theorem statement is the bug.

Case (2026-04-26). We submitted a candidate `wald_wolfowitz_optimal` theorem. Aristotle returned a closure that combined a Bool partition with a log-boundary constraint to force both $\alpha \geq 1$ and $\alpha < 1$. The kernel accepted the proof. The audit flagged the hypothesis chain as self-contradictory. We rejected the closure, restated the theorem in two-measure form (separating the null and alternative measures explicitly), and resubmitted. The two-measure form was the intended target. The original Bool-partition form was a statement bug.

Audit policy. The library treats vacuous closures as theorem-statement bugs, not as prover successes. Concretely: every Aristotle-returned proof is audited for contradictory premises before integration. Closures flagged as vacuous block the corresponding theorem from the axiom-clean library until the statement is reformed. This policy is not unique to Aristotle. The same audit applies to any external prover, including human proofs that close via a hypothesis contradiction the author did not notice.

4.3. Z3 as a second oracle

The `z3_check` tactic dispatches linear-real-arithmetic goals to Z3 (de Moura & Bjørner, 2008). Z3 returns a verdict (unsat of the negated goal). The verdict is then reconstructed inside Lean by `linarith`. The Lean kernel is the final word. The pattern is the CoqHammer (Czajka & Kaliszyk, 2018) reconstruction strategy adapted to Lean 4. Z3 cannot smuggle axioms. `linarith` is axiom-clean and the reconstruction step certifies the witness inside the kernel. If `linarith` cannot reconstruct, the `z3_check` call fails loud rather than silently trusting Z3.

4.4. When to call Aristotle

The library convention is to attempt local closure first. If the goal closes by `rfl`, `simp`, `positivity`, `linarith`, `nlinarith`, `polyrith`, or the registered `stat_lemma` database, no Aristotle call is made. Aristotle is reserved for hard structural Mathlib gaps: measure-theoretic σ-additivity steps, dominated-convergence side conditions, matrix-concentration intermediate lemmas, and similar obligations for which a hand-Mathlib closure either does not exist or would require an order of magnitude more dispatch. The vacuous-closure audit is the gate on integration. The dispatch threshold is only the call decision.

5. Comparison and discussion

5.1. Comparison to existing automation

aesop. Aesop (Limperg & From, 2023) is the workhorse forward-search tactic in Lean 4. `pythia` declares two named aesop rule sets (`Pythia` and `Pythia.AnytimeValid`) and uses aesop in stage 1 of the dispatcher chain. `pythia` is not a replacement for aesop. It is a statistics-domain registry plus a Z3 oracle plus a vacuous-closure audit, layered on top of aesop.

Sledgehammer. Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) dispatches Isabelle/HOL goals to external ATPs (Vampire, E, Z3, CVC4) and reconstructs the proof. There is no direct Lean 4 equivalent. `pythia` addresses the statistics-domain case via `z3_check` for linear-real-arithmetic and Aristotle as a structural oracle.

A general-purpose Lean Sledgehammer is out of scope for this paper.

CoqHammer. CoqHammer (Czajka & Kaliszyk, 2018) dispatches Coq goals to external ATPs with kernel-checked reconstruction. `z3_check` in `pythia` adopts the same reconstruction template (witness from external prover, in-kernel checking via `linarith`). CoqHammer is the design template. We do not claim a Lean-side reimplementation of CoqHammer's full premise selection.

Mathlib's existing tactic suite. Mathlib supplies `linarith`, `polyrith`, `nlinarith`, `positivity`, `measurability`, `bound`, `gcongr`. None of these is a statistics-domain hammer. `stats_ineq` is the closest `pythia` relative of the Mathlib `bound` family. The `@[stats_ineq]` attribute re-tags lemmas as `@[bound]`, so the mature `bound` dispatch picks them up automatically. `pythia` composes with all of the listed Mathlib tactics. The fall-through chain at stage 2 of the dispatcher combines `simp`, `omega`, `linarith`, and `positivity`. The chain is the explicit composition.

5.2. Aristotle vs DSPv2 vs Opus 4.6 (illustrative)

Table 2 reports four FormalAVS problems (Anonymous, 2026) on which Aristotle, DSPv2 (DSPv2-7B GPTQ-int8, a quantised local-GPU prover), and Opus 4.6 (an API generalist drafter) were each given a single-attempt protocol (Aristotle is one session per problem. DSPv2 and Opus run pass@5 at $N = 5$ on the 48-slate.) The full benchmark numbers are in the dataset paper. The point of this subset is to expose the four solver regimes that the benchmark tail surfaces. Per-problem solve outcomes are kernel-checked.

Problem	Aristotle	DSPv2	Opus 4.6
`betting_is_kelly_optimal`*	1/1	n/a	n/a
etavector‡	1/1	1/5	0/5
`c_sharp_ranking`	1/1	0/5	0/5
`equivalence_break_at _finite_precision`	0/3†	0/5	0/5

Table 2. Illustrative four-problem subset of FormalAVS. Each cell is *(closures)/(attempts)*. Aristotle is one session per problem. DSPv2 and Opus are pass@5 at N=5. The four rows expose four distinct solver regimes. Universal closure sits at the top. Majority closure is `etavector`, the $\sqrt{2}$ identity that Kimi and Mistral also close at N=5. Aristotle-monopoly is `c_sharp_ranking`. The solver-unreached frontier is `equivalence_break`. *Companion-archive entry, not in the 48-slate drafter sweep, included for the universal regime. †Aristotle returned a counterexample on the as-stated form (missing symmetry hypothesis). The corrected form remains open. ‡Lean lemma name shortened from the full *etavector_eq_sqrt_two_mul_etahr*, see Anonymous (2026). Full per-tier benchmark numbers in Anonymous (2026).

The wider table on the full benchmark subset is in Appendix C.

5.3. Limitations

Three concrete gaps. (i) Domain scope: `pythia` is statistics-domain only. It does not address general proof automation in Lean 4. (ii) Toolchain pin: the library ships pinned to Lean 4.28 + Mathlib v4.28.0. Mathlib evolves quickly. Each major Mathlib release typically requires a porting pass. (iii) Aristotle dependency: the hardest closures route to a proprietary external prover. The `z3_check` oracle is open-source. The Aristotle oracle is not. The library is usable without Aristotle for any goal that closes via the `pythia` dispatcher chain or `z3_check`. A subset of structural Mathlib gaps requires either Aristotle or a hand proof.

References

Anonymous. FormalAVS: a benchmark for statistical theorem proving in Lean 4, 2026. Companion dataset paper, anonymized for double-blind review.

Blanchette, J. C., Böhme, S., and Paulson, L. C. Extending Sledgehammer with SMT solvers. In *23rd International Conference on Automated Deduction (CADE)*, pp. 116–130, 2011. doi: 10.1007/978-3-642-22438-6_11.

Czajka, Ł. and Kaliszyk, C. Hammer for Coq: Automation for dependent type theory. *Journal of Automated Reasoning*, 61(1-4):423–453, 2018. doi: 10.1007/s10817-018-9458-4.

de Moura, L. and Bjørner, N. Z3: An efficient SMT solver. In *14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 337–340, 2008. doi: 10.1007/978-3-540-78800-3_24.

de Moura, L. and Ullrich, S. The Lean 4 theorem prover and programming language. In *28th International Conference on Automated Deduction (CADE)*, pp. 625–635, 2021. doi: 10.1007/978-3-030-79876-5_37.

Harmonic. Aristotle: an automated theorem prover for Lean. `https://harmonic.fun/`, 2025. Proprietary commercial tool.

Howard, S. R., Ramdas, A., McAuliffe, J., and Sekhon, J. Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2):1055–1080, 2021. doi: 10.1214/20-AOS1991.

Lieb, E. H. Convex trace functions and the Wigner-Yanase-Dyson conjecture. *Advances in Mathematics*, 11(3):267–288, 1973. doi: 10.1016/0001-8708(73)90011-X.

Limperg, J. and From, A. H. Aesop: White-box best-first proof search for Lean. In *Proceedings of the 12th ACM SIGPLAN International Conference on Certified*

Programs and Proofs (CPP), pp. 253–266, 2023. doi: 10.1145/3573105.3575671.

Paulson, L. C. and Blanchette, J. C. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL)*, 2010.

Ramdas, A., Grünwald, P., Vovk, V., and Shafer, G. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science*, 38(4):576–601, 2023.

Rockafellar, R. T. and Uryasev, S. Optimization of conditional value-at-risk. *Journal of Risk*, 2(3):21–41, 2000.

Shin, J., Ramdas, A., and Rinaldo, A. E-detectors: a nonparametric framework for sequential change detection. *New England Journal of Statistics in Data Science*, 2024.

The Mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pp. 367–381, 2020. doi: 10.1145/3372885.3373824.

The Mathlib Community. Mathlib: the Lean 4 mathematical library. `https://github.com/leanprover-community/mathlib4`, 2025. Repository tagged at v4.28.0 for the pythia pin.

Tropp, J. A. User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434, 2012. doi: 10.1007/s10208-011-9099-z.

Waudby-Smith, I. and Ramdas, A. Estimating means of bounded random variables by betting. *Journal of the Royal Statistical Society Series B*, 86(1):1–27, 2024.

Whitehouse, J., Ramdas, A., Wu, Z. S., and Sutton, R. Time-uniform self-normalized concentration for vector-valued processes. 2025.

Wold, H. *A Study in the Analysis of Stationary Time Series*. Almqvist & Wiksell, Stockholm, 1938.

A. Library inventory

The shipped library at the AI4MATH 2026 submission cut spans the modules below. Each line lists module path and one-line summary. All listed modules are axiom-clean against $\mathcal{A}_{\text{core}} = \{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}$ unless explicitly flagged as scaffold.

A.1. Anytime-valid inference

- `Pythia.HowardRamdasCS`: Howard-Ramdas Ville bound for self-normalised confidence sequences (rate $\eta_{\text{HR}}(b) = \sqrt{b \log 2}$). Axiom-clean.
- `Pythia.BettingCS`: betting-CS coverage certificate (rate $\eta_{\text{betting}}(b) = 1/\sqrt{b \log 2 + 1}$). Axiom-clean.
- `Pythia.BettingStrategy`: betting-strategy admissibility scaffolding for the `anytime_valid` dispatch.
- `Pythia.VilleSupermartingale`: countable-time and finite-horizon Ville inequalities. The `ville_supermartingale`, `ville_ineq`, and `ville_supermartingale_finite` lemmas are the workhorse.
- `Pythia.EDetector`: e-process formalism, with `combine_eprocesses_avg` (average-merge) as the workhorse closer. Anytime-valid Type-I error control via Markov-at-stopping.

A.2. Concentration

- `Pythia.SubGaussianMG`: sub-Gaussian martingale concentration. The Chernoff exponent $\exp(-\tau^2/(2\sigma^2 N))$ bound. Axiom-clean.
- `Pythia.SubGamma`: sub-gamma tail bound. Axiom-clean.
- `Pythia.MGFBoundedSubGamma`: bounded-RV Bernstein-Bennett MGF bound, $\mathbb{E}[\exp(\lambda X)] \le \exp(\sigma^2\lambda^2/(2(1 - b\lambda/3)))$ for centred $|X| \le b$. Axiom-clean.
- `Pythia.Bernstein`: iid and martingale Bernstein applications.

A.3. Matrix concentration

- `Pythia.MatrixLieb`: Lieb concavity step (Lieb, 1973) and operator-monotone log composition. Scaffold. Dependent operator-trace inequalities route to Aristotle.
- `Pythia.MatrixBernstein`: iid matrix-Bernstein head theorem (Tropp, 2012). Scaffold.
- `Pythia.MatrixBernsteinFull`: martingale extension. Scaffold.

A.4. Other modules

- The `Pythia.Control` subpackage carries `LyapunovDiscrete`, the discrete-time Lyapunov stability result for linear systems.
- `Pythia.Queueing.ErlangB`: Erlang-B blocking formula.

- `Pythia.Queueing.LittlesLaw`: Little's law steady-state identity.
- `Pythia.TimeSeries.WoldDecomposition` (Wold, 1938).
- `Pythia.Risk.CVaR` (Rockafellar & Uryasev, 2000): the Rockafellar-Uryasev variational characterisation of CVaR.
- `Pythia.Asymptotics.DeltaMethod`, `Pythia.Asymptotics.DeltaMethodMulti`: scalar and multivariate delta method.
- `Pythia.InfoTheory.BretagnolleHuberBinary`: binary-case total-variation bound via the Bhattacharyya bridge.
- The `Pythia.MeasureTheory` subpackage carries `OptionalStoppingUnbounded` and `PathMeasureRN`. Both are structural support for the Ville chain.

B. Worked examples

Each example below is a literal regression test from the `pythia` test suite. Every `example` compiles axiom-clean against $\mathcal{A}_{\text{core}}$.

B.1. anytime_valid: Ville-bound closer

The countable-time variant closes the canonical Ville maximal inequality on a non-negative supermartingale.

```
import Pythia.Tactic.AnytimeValid
open MeasureTheory ProbabilityTheory
    ENNReal Pythia

example {Omega : Type*} {m0 :
    MeasurableSpace Omega}
   {mu : Measure Omega} [IsFiniteMeasure mu
       ]
   {f : Nat -> Omega -> Real}
   {F : Filtration Nat m0}
   (hsup : Supermartingale f F mu)
   (hnn : forall t omega, 0 <= f t omega)
   (hint : Integrable (f 0) mu)
   {c : Real} (hc : 0 < c) :
   mu {omega | exists t : Nat, f t omega >=
        c}
     <= (Integral.integral mu (f 0)).
         toNNReal / c.toNNReal := by
  anytime_valid
```

The tactic dispatches via the `ville_supermartingale` exact path. Hypotheses are discharged by `assumption`. No registered rule is consulted. The kernel-checked term reduces under {`propext`, `Classical.choice`, `Quot.sound`}.

B.2. z3_check: linear-real-arithmetic with reconstruction

A three-step linear chain over $\mathbb{R}$. Z3 is queried as a ranking filter. `linarith` produces the actual proof term.

```
import Pythia.Tactic.Z3Check
open Pythia

example {a b c : Real} (h1 : a < b) (h2 : b
     <= c) :
   a < c := by
  z3_check
```

Trace: the encoder produces the SMT-LIB query `(set-logic QF_LRA) ... (assert (< v_a v_b)) (assert (<= v_b v_c)) (assert (not (< v_a v_c))) (check-sat)`, which Z3 returns `unsat`. `linarith` then constructs the Farkas certificate inside Lean. Z3 cannot smuggle axioms. If `linarith` disagreed, the tactic would fail loud.

B.3. pythia: chain-rule fall-through to aesop

A goal with no matching `@[stat_lemma]` but a closed-form chain in `linarith`. Stage 1 (aesop with the `Pythia` ruleset) declines. Stage 2 (the cleanup chain combining `simp`, `omega`, `linarith`, and `positivity`) closes.

```
import Pythia.Tactic.Pythia
open Pythia

example (a b c d : Real)
    (h1 : a <= b) (h2 : b <= c) (h3 : c <= d
        ) :
   a <= d := by
  pythia
```

The dispatcher's `first`-combinator tries aesop, observes no applicable `@[stat_lemma]` closer, then tries the cleanup chain. `linarith` closes the chain in one step.

B.4. Registry-extend: tagging a user theorem

User extensions consist of tagging library lemmas with the appropriate attribute. Below: a trivial real-arithmetic identity registered for `pythia` via `@[stat_lemma]`.

```
import Pythia.Tactic.Pythia
open Pythia

@[stat_lemma]
theorem add_zero_real (x : Real) : x + 0 =
    x := by ring

-- pythia now closes goals matching the
    registered lemma:
example (x y : Real) : (x + 0) + (y + 0) =
    x + y := by
  pythia
```

The attribute re-elaborates as `@[aesop safe apply (rule_sets := [Pythia])]`. The lemma then joins the `Pythia` aesop ruleset and is consulted by `pythia` on every subsequent invocation. No fork or rebuild of the tactic implementation is required.

C. Aristotle vs DSPv2 vs Opus 4.6 (wider subset)

Table 3 reports a wider subset of FormalAVS (Anonymous, 2026) per-tier closure rates for the three solver families. Aristotle is one-attempt-per-target. DSPv2 and Opus are pass@5 at $N{=}5$ on the 48-target slate (T0 through T2 plus T3). Cells are *(closures)/(targets)* at the tier denominator, with T4 (library-gap) and T5 (capability-ceiling) appended.

Solver	T0	T1	T2	T3	T4	T5
Aristotle	7/7	33/34	7/8	6/6	0/4	0/1
DSPv2-7B GPTQ-int8	3/7	8/34	0/8	1/6	0/4	0/1
Opus 4.6	0/7	13/34	0/8	0/6	0/4	0/1

Table 3. Wider per-tier comparison from FormalAVS. Aristotle: one attempt per target. DSPv2 and Opus: pass@5 at $N{=}5$. T4 is solver-hit-Mathlib-gap (every solver $0/4$ for the same library-gap reason). T5 is the all-solver capability-ceiling target. The 6/6 vs 1/6 vs 0/6 column at T3 (cross-function inequality wall) is the benchmark's cleanest solver-specific signal. The single DSPv2 T3 closure is the $\eta_{\text{VECTOR}}{=}\sqrt{2}\,\eta_{\text{HR}}$ identity (the *etavector* target). Kimi K2.5 and Mistral Large 3 also close this one in the wider drafter sweep. Source: dataset paper (Anonymous, 2026), Tables 1-2 reproduced verbatim.

Per-target capability-wall tail. Table 4 reports the per-target closure matrix on the five capability-wall tail problems used in the dataset paper to classify solver regimes.

Target	Aristotle	Kimi K2.5	Sonnet 4.6	Gemini 3 Pro
betting_is_kelly_optimal*	1/1	1/1	1/1	1/1
c_vector_eq_sqrt_two_mul_c_HR	1/1	1/1	0/5	1/2
c_sharp_ranking	1/1	0/9	0/5	1/3
quantized_entropy_bound	1/1	0/9	0/5	0/3
equivalence_break_at _finite_precision	0/3†	0/19	0/15	0/13

Table 4. Per-target capability-wall tail. Cells are *(closures)/(attempts)*. Five distinct solver regimes appear: universal closure, majority closure, drafter-monopoly (Gemini's gcongr-prior closes c_sharp_ranking), Aristotle-monopoly, and solver-unreached. Source: Anonymous (2026), Table 4. *Companion-archive entry, not in the 48-slate drafter sweep. †Aristotle: two sessions timed out at the 20-minute compute gate. One returned a counterexample on the as-stated form (missing symmetry hypothesis, (Anonymous, 2026) §Aristotle-refute).

D. Reproduction

Toolchain pin. Lean 4.28 + Mathlib v4.28.0. The repository's `lean-toolchain` file declares the exact toolchain. The Mathlib dependency in `lakefile.toml` pins the v4.28.0 release tag.

Build. The library URL is redacted for double-blind review (full URL in camera-ready). Reproduction commands once the URL is supplied:

```
git clone <pythia-url-redacted>
cd pythia
lake exe cache get
lake build
```

Axiom audit. Each shipped module ends with a `#print axioms` block. The expected audit set is {`propext`, `Classical.choice`, `Quot.sound`}. Any deviation fails the audit.

FormalAVS benchmark. The full benchmark is reported in Anonymous (2026). Reproduction commands for the benchmark are in that paper's reproduction appendix. The benchmark URL is redacted for double-blind review.

E. Related work, extended

E.1. Lean 4 tactic infrastructure

Mathlib ships a tactic suite the `pythia` dispatcher composes with. `linarith` produces Farkas certificates and `polyrith` routes polynomial-ideal goals via Sage with kernel-checked reconstruction. `nlinarith` extends `linarith` to a small non-linear fragment. Sign goals route through `positivity` and σ-algebra-membership goals through `measurability`. Generalised-congruence monotonicity is discharged by `gcongr`. Monotone-composition goals route through `bound`, which uses an attribute-driven rule database. The closest Mathlib relative to `pythia` is `bound`: the `@[stats_ineq]` attribute literally re-tags lemmas as `@[bound]`, so the underlying `bound` dispatch picks them up automatically. Aesop (Limperg & From, 2023) supplies the `rule_sets` mechanism the `pythia` dispatcher uses. `pythia` is therefore a thin domain-registration layer over `aesop` plus a Z3 oracle.

E.2. Sledgehammer (Isabelle)

Sledgehammer (Paulson & Blanchette, 2010; Blanchette et al., 2011) dispatches Isabelle/HOL goals to Vampire, E, Z3, CVC4, with proof reconstruction inside Isabelle's kernel via Metis or `smt`-tactic replay. The reconstruction discipline is the template for both CoqHammer and `z3_check`. There is no Lean 4 equivalent at the time of writing. `pythia` does not attempt the Isabelle Sledgehammer's full premise-selection front end. The library ships only the linear-real-arithmetic instance via `z3_check`.

E.3. CoqHammer (Coq)

CoqHammer (Czajka & Kaliszyk, 2018) dispatches Coq goals to Vampire, E, Z3, CVC4 with kernel-checked reconstruction. Premise selection in CoqHammer is k-NN over a featurised representation of the Coq context. The reconstruction step is a small certifying tactic (`sauto`) that replays the ATP's witness inside the Coq kernel. `z3_check`'s reconstruction via `linarith` mirrors this template at small scale. The external prover supplies the verdict and the in-kernel tactic supplies the proof term. The kernel is the final arbiter. We do not claim a full Lean-side reimplementation of CoqHammer's premise-selection layer.

E.4. Statistical formalisation libraries

Mathlib's `Probability.Martingale` module supplies Doob decomposition and Azuma's inequality but does not instantiate the canonical confidence-sequence families, the per-family rate functions, or Ville's inequality for the betting construction. The `pythia` library closes that gap. The FormalAVS benchmark (Anonymous, 2026) measures the result. Coq's mathcomp.analysis and the Isabelle/HOL probability formalisations cover overlapping but disjoint sub-areas, and are not attribute-driven hammers.